

Name- Palak Chanchlani
Class - D20A
Roll no.: 12

Experiment 5

Aim - Deploying a Voting/Ballot Smart Contract

Theory :

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement is used as a safeguard to validate conditions before executing the rest of the function. If the condition fails, the transaction is reverted and no state changes are saved. This ensures that invalid actions do not corrupt contract data.

For example, in a voting contract:

- Checking if the caller is the chairperson before granting rights.
- Ensuring a voter has not already voted.

Confirming that a voter has the right to vote before allowing them to cast a ballot.

- Thus, require statements enforce correctness, security, and prevent misuse of contract functions.

2. Keywords: mapping, storage, and memory

mapping:

- Mappings are key-value data structures in Solidity, often used to associate addresses with voter information. They are efficient for lookups but cannot be iterated directly. For example:

solidity

```
mapping(address => Voter) public voters;
```

This links each Ethereum account to its voter details.

storage:

- Storage refers to permanent contract data saved on the blockchain. Variables declared at contract level are stored here. Since writing to storage consumes gas, it is costly but persistent. Example: voter details stored in the voters mapping remain available throughout the contract lifecycle.

memory:

- Memory is temporary data storage used only during function execution. It is cheaper than storage and discarded once the function ends. For instance, proposal names passed to the constructor are stored in memory during execution.

3. Why bytes32 Instead of string?

Earlier versions of the Ballot contract used bytes32 for proposal names because it is fixed-size, cheaper in gas, and faster for comparisons. However, it restricts names to 32 characters.

On the other hand, string is dynamic and user-friendly, allowing longer names but consuming more gas.

Modern contracts often prefer string for readability, even though bytes32 remains useful when efficiency is the priority.

Code :

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }

    struct Proposal {
        string name; // proposal name (string for readability)
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;
    mapping(address => Voter) public voters;
    Proposal[] public proposals;

    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }
}
```

```

function giveRightToVote(address voter) public {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right");
    voters[voter].weight = 1;
}

```

```

function delegate(address to) public {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted");
    require(to != msg.sender, "Self-delegation is disallowed");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

```

```

function vote(uint proposal) public {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted");
    sender.voted = true;
    sender.vote = proposal;
    proposals[proposal].voteCount += sender.weight;
}

```

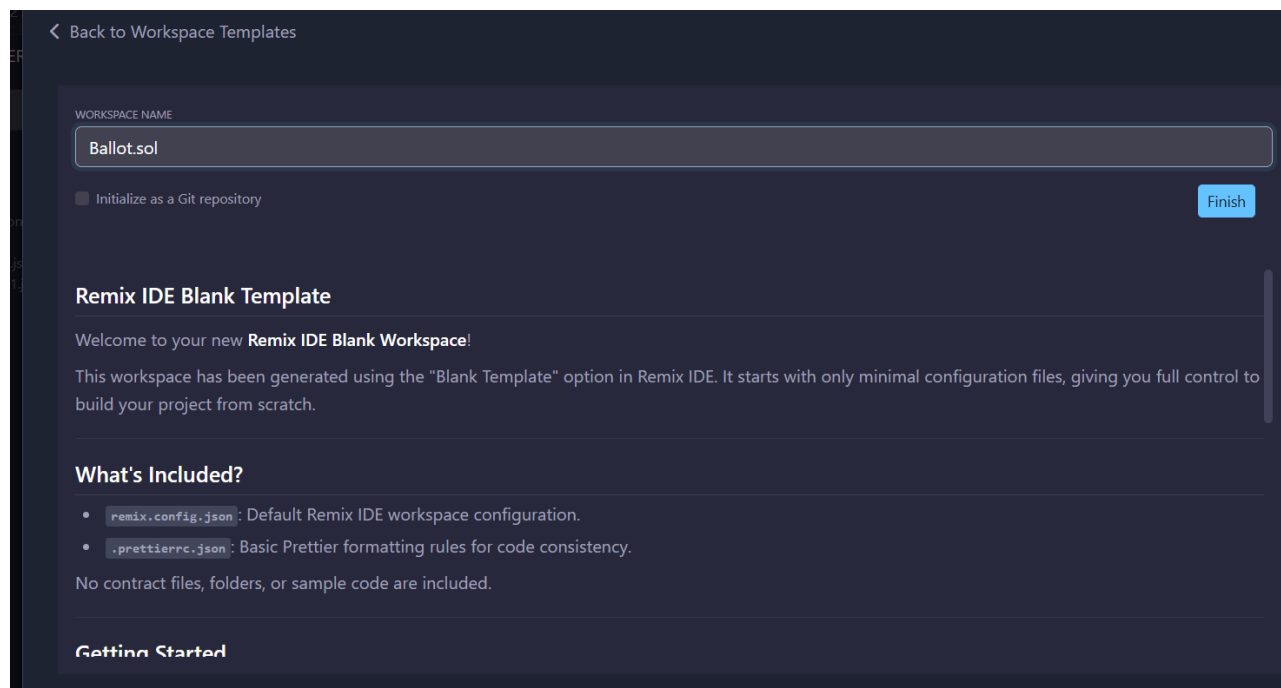
```

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;
    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() public view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}
}

```

Output :



REMIX2.0.2

Ballot.sol

DEPLOY & RUN TRANSACTIONS

Environment

ForkReset

Remix VM

Osaka

Account 1

0x5B3...eddC4

99.999 ETH

Ballot

Ballot.sol

Compiled

proposalNames

string[]

["Palak"]

Call data

Parameters

Value

0

wei

Gas limit

auto

0

Deploy

Deployed Contracts 1

Add Contract

Compiled

Home

Ballot.sol

.prettierrc.json

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // voting weight
11         bool voted; // has the voter voted
12         address delegate; // to whom the vote is delegated
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
17         string name; // proposal name (changed from bytes32 to string)
18         uint voteCount; // number of votes received
19     }
20
21     address public chairperson;
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

• ethers.js

Type the library name to see available commands.

✓

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xfa5...c9692

Debug

0x5B3...eddC4

99.999 ETH

Low level interaction

non-payable

giveRightToVote

address

0xd9145CCE52D386f254917e481eB44e9943F39138

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

9

struct Voter {

10

uint weight; // voting weight

11

bool voted; // has the voter voted

12

address delegate; // to whom the vote is delegated

13

uint vote; // index of the voted proposal

14

}

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

• Execute JavaScript scripts:

- Input a script directly in the command line interface

- Select a Javascript file in the file explorer and then run "remix.execute()" or "remix.executeCurrent()" in the command line interface

- Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:

• ethers.js

Type the library name to see available commands.

✓

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xfa5...c9692

Debug

✓

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...39138 logs: 0 hash: 0xf71...b911c

Debug

cam Alert

Initialize as git repo

Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Osaka

Account 3

100.000 ETH

0x4B2...C02db

Deploy

Remix VM osaka

Ballot

Ballot.sol

Compiled

proposalNames

string[]

["Palak"]

Call data

Parameters

Value

0

wei

Gas limit

auto

0

Deploy

Compile

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
```

Explain contract

0

Listen on all

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

transact to Ballot.giveRightToV

revert

The transaction has been

Reason provided by the contract

If the transaction failed for not

Error occurred: revert.

revert

The transaction has been

Reason provided by the contract

If the transaction failed for not

If the transaction failed for not

[vm] from: 0x5B3...ed

value: 0 wei data: 0x9

Scan Alerts Initialize as git repo Did you know? To protect on s

Environment

Fork

Reset

Remix VM

Osaka

Account 3

99.999 ETH

0x4B2...C02db

call proposals uint256

call voters address

call winnerName

Low level interaction

call winnerName

ostring: Palak

Call

Balance

0 ETH

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
```

Explain contract

0

Listen on all transactions

Filter with transaction hash or ad...

transact to Ballot.giveRightToVote errored: Error occurred: Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Voter already has right".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Voter already has right".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

If the transaction failed for not having enough gas, try increasing the gas limit gently..

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138

value: 0 wei data: 0x9e7...c02db logs: 0 hash: 0x488...ddf3c

Debug

[vm] from: 0x4B2...C02db to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei

data: 0x812...00000 logs: 0 hash: 0x3a0...b8e7f

Debug

[call] from: 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db

to: Ballot.winnerName() data: 0xe2b...a53f0

Debug

Conclusion

In this experiment, a Voting/Ballot smart contract was successfully deployed and executed using Solidity on the Remix IDE. The role of require statements was observed in maintaining security and correctness by restricting unauthorized actions and preventing duplicate voting. The use of mapping helped in linking voter addresses with their details, while the distinction between storage and memory highlighted how Solidity manages permanent and temporary data efficiently. The difference between bytes32 and string for proposal names was also understood, showing the trade-off between gas optimization and user-friendly readability. Overall, the experiment demonstrated how blockchain smart contracts can enforce transparent voting rules, ensure fairness, and declare results securely on the Ethereum platform.